CLAUDE CODE SOURCE STUDY · VOL. 02

실행 표면과 상태 소유권

REPL, print/headless, QueryEngine, SDK와 remote가 실제로 어디에서 갈라지고 합쳐지는지 추적한다

> 프로젝트의 구조를 외우는 대신, 코드가 어떤 상태와 불변조건을 지키며 결정적인 동작을 보장하는지 이해하기 위한 스터디 보고서다.

이 권에서 얻게 될 통찰

이 권을 마치면 QueryEngine을 전체 제품의 단일 엔진으로 오해하지 않고 각 상태의 실제 owner를 설명할 수 있다.

Fixed source
Claude Code v2.1.88 복원 snapshot
2ca5ddabfed5f220812ea11f029eda03b21bc4c1
2026-09-20 · Korean study edition

READING BASELINE

OBSERVED

먼저, 지금 보고 있는 시스템의 상태를 고정한다

CLI entrypoint는 여러 fast path를 가진다. 일반 대화에서도 interactive REPL과 headless 계열은 서로 다른 orchestration shell을 사용한다.

구현에서 직접 확인되는 범위

REPL은 query()를 직접 호출하고, ask()는 호출마다 QueryEngine을 생성한다. query()와 Tool runtime이 핵심 공유 영역이다.

운영을 말할 때 남겨야 할 경계

각 surface는 permission interaction, backpressure, UI state, message persistence를 다르게 다룬다.

이 보고서가 추측으로 채우지 않는 것

공식 SDK host topology 전체, 누락 remote/daemon module의 동작, 그리고 각 surface의 production 사용 비율은 현재 snapshot만으로 확정할 수 없다. 이후의 설명은 이 공백을 유지한 채, 실제 코드가 보여 주는 구조와 보장만을 다룬다.

읽는 동안 반복할 질문

이 문장은 코드로 관찰한 사실인가, 구조에서 끌어낸 해석인가, 아니면 개선을 위한 제안인가?

SYSTEM READING MAP

OBSERVED

파일이 아니라 책임과 보장의 연결로 읽는다

이 지도는 번호를 따라 구성요소를 외우게 하지 않는다. 각 영역이 어떤 의미를 소유하고, 그 결과가 다음 영역의 판단 근거가 되며, 마지막에 어떤 외부 보장으로 닫히는지를 문장으로 연결한다.

진입 경계

Process `cli.tsx`

fast path와 dynamic import

"fast path와 dynamic import"가 확인되어야 다음의 "Composition" 영역이 "config/auth/dependency"를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.

상태 소유

Composition `main.tsx`

config/auth/dependency

"config/auth/dependency"가 확인되어야 다음의 "Interactive shell" 영역이 "UI + application controller"를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.

진입 경계

Interactive shell `REPL.tsx`

UI + application controller

"UI + application controller"가 확인되어야 다음의 "Headless shell" 영역이 "queue + one-shot engine"를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.

진입 경계

Headless shell `print.ts + ask()`

queue + one-shot engine

"queue + one-shot engine"가 확인되어야 다음의 "Session abstraction" 영역이 "headless/SDK state owner"를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.

상태 소유

Session abstraction `QueryEngine`

headless/SDK state owner

"headless/SDK state owner"가 확인되어야 다음의 "Shared core" 영역이 "turn protocol"를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.

의미 연결

Shared core `query()`

turn protocol

이 책임은 앞선 판단을 외부에서 관찰 가능한 결과로 닫는다. 이 지점의 실패 처리가 모호하면 앞의 모든 단계가 성공했더라도 시스템 전체는 정산되지 않은 상태로 남는다.

선을 따라갈 때에는 구현 이름보다 인접한 두 책임 사이에서 전달되는 identity, state, completion을 확인한다. 그 셋 중 하나가 사라지는 지점이 구조적 결함이 나타날 가능성이 가장 높은 경계다.

01 프로세스가 실행 표면을 선택한다

모드 선택은 출력 형식 선택이 아니라 process lifetime과 dependency graph 선택이다.

이 장의 논증은 다음 질문을 차례로 해결한다

알고리즘	진입 경계 우선순위 기반 CLI route resolution	코드	책임 연결 Dynamic import가 architecture boundary 역할을 한다
해석	상태 소유 main.tsx는 composition root이면서 결합 압력점이다	코드	진입 경계 launchRepl은 얇지만 REPL은 얇지 않다
구분	책임 연결 실행 모드별 계약		

코드 독해의 시작점

`src/entrypoints/cli.tsx` → `src/main.tsx` → `src/replLauncher.tsx`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

ALGORITHM A03 · 독자의 질문

우선순위 기반 CLI route resolution 은 어떤 시스템 문제를 닫는가

하나의 binary가 version 출력부터 remote·daemon·일반 CLI까지 여러 실행 표면을 제공한다. 사용자는 같은 명령을 실행하지만 실제로는 첫 predicate가 선택한 서로 다른 module graph로 들어간다.

코드가 필요해지는 이유

하나의 binary에서 version, MCP host, remote, daemon, background, 일반 CLI를 충돌 없이 선택한다.

OBSERVED — src/entrypoints/cli.tsx의 top-level dispatcher가 argv와 compile-time gate를 읽어 오직 하나의 route를 선택한다.

INFERENCE — entrypoint는 각 subsystem의 동작을 소유하지 않고, 어느 subsystem이 process lifetime을 소유할지를 먼저 결정한다.

이 절에서 답할 결정 질문

route 순서가 왜 제품 의미와 startup cost를 동시에 결정하며, dynamic import가 그 결정을 어떻게 보존하는가?

이 설명을 확인할 코드 src/entrypoints/cli.tsx:28-93

ALGORITHM A03 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/entrypoints/cli.tsx의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01

short-circuit chain과 lazy module loading

앞 predicate가 참이면 뒤 predicate는 평가·실행되지 않는다. dynamic import는 선택되지 않은 route의 module initialization도 늦춰 startup과 dead-code elimination 경계를 만든다.

> **실제 코드로 돌아가면.** 28–93행에서 predicate 순서, early return, import 위치를 한 묶음으로 읽어야 한다.

02

compile-time gate와 runtime policy의 차이

bundle에서 코드를 제거하는 literal과 실행 중 허용 여부를 판단하는 policy는 서로 다른 시간축이다.

> **실제 코드로 돌아가면.** feature 값이 false일 때 route가 단순 비활성인지, module graph에서 제거 가능한지 구분한다.

이론은 정답이 아니라 코드에 던질 질문이다

상태·계산 모델
`chain of responsibility`
단계의 순서나 표현을 바꿔도 같은 상태와 불변조건에 도달하는가?

구성·변형·성능
`lazy loading · partial evaluation`
조립 방식이나 최적화를 바꿔도 핵심 계약이 adapter와 build 변형 안에서 보존되는가?

ALGORITHM A03 · 최소 예제

겹치는 predicate가 있을 때 첫 번째 route가 process를 소유한다

두 조건이 동시에 참일 수 있는 작은 dispatcher를 통해 순서가 단순 표현이 아니라 precedence임을 본다.

설명을 위한 최소 실행 예제

```
async function dispatch(args) {
  if (args.includes('--version')) return 'version'
  if (args.includes('--remote')) return await import('./remote.js')
  return await import('./interactive.js')
}

dispatch(['--version', '--remote']) // 'version'
```

관찰 시점	값과 상태	프로젝트 코드에서의 의미
입력 해석	두 flag가 모두 존재한다.	registry의 두 항목이 아니라 첫 predicate가 authority를 가진다.
첫 branch	version이 terminal result를 반환한다.	remote module은 load되지 않는다.
순서 변경	remote predicate를 앞으로 옮기면 결과와 module graph가 바뀐다.	리팩터링은 동작 변경으로 검토해야 한다.

예제를 실제 구현에 다시 연결한다

실제 dispatcher에서도 fast path의 순서와 각 branch의 return을 characterization test로 고정해야 한다.

ALGORITHM A03 · 실제 코드

OBSERVED

우선순위 기반 CLI route resolution 의 결정점은 어디에 있는가

entrypoint는 ordered early-return dispatcher다.

입력 표현. process.argv, compile-time feature 값, 그리고 환경 변수

보존되는 작업 상태. 아직 load되지 않은 module graph, 그리고 startup profiler checkpoint

fast-path dispatcher

src/entrypoints/cli.tsx:28-93

```
28    /**
29     * Bootstrap entrypoint - checks for special flags before loading the full CLI.
30     * All imports are dynamic to minimize module evaluation for fast paths.
31     * Fast-path for --version has zero imports beyond this file.
32     */
33    async function main(): Promise<void> {
34      const args = process.argv.slice(2);
35
36      // Fast-path for --version/-v: zero module loading needed
37      if (args.length === 1 && (args[0] === '--version' || args[0] === '-v' || args[0] === '-V')) {
38        // MACRO.VERSION is inlined at build time
39        // biome-ignore lint/suspicious/noConsole:: intentional console output
40        console.log(`${MACRO.VERSION} (Claude Code)`);
41        return;
… 49 lines omitted; use the source link for the complete range …
91        await runComputerUseMcpServer();
92        return;
93      }
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A03 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

process.argv, compile-time feature 값, 그리고 환경 변수은 처음부터 신뢰된 실행 상태가 아니다. 아직 load되지 않은 module graph, 그리고 startup profiler checkpoint가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** 가장 값싼 --version을 먼저 검사한다. 이 결과가 다음 판단 "특수 process flag를 고정 순서로 검사한다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** 특수 process flag를 고정 순서로 검사한다. 이 결과가 다음 판단 "필요한 path에서만 dynamic import한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** 필요한 path에서만 dynamic import한다. 이 결과가 다음 판단 "일치하지 않으면 일반 application bootstrap으로 이동한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** 일치하지 않으면 일반 application bootstrap으로 이동한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

하나의 실행에서 정확히 한 top-level route만 선택된다. fast path는 불필요한 module side effect를 평가하지 않는다. feature-excluded path는 외부 bundle에서 사라질 수 있다.

ALGORITHM A03 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **상태·계산 모델의 질문으로 읽기.** 하나의 실행에서 정확히 한 top-level route만 선택된다.
- **구성·변형·성능의 질문으로 읽기.** fast path는 불필요한 module side effect를 평가하지 않는다.
- **상태·계산 모델의 질문으로 읽기.** feature-excluded path는 외부 bundle에서 사라질 수 있다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- predicate 순서 충돌
- top-level import의 조기 side effect
- gate와 runtime policy의 불일치

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED registry보다 명시적 early return을 택해 startup cost와 DCE 가능성을 우선한다.

RECOMMENDATION route table을 문서화하되 dynamic import와 compile-time literal을 훼손하지 않는 characterization test를 둔다.

ALGORITHM A03 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

선택된 route만 자신의 초기화와 event loop lifetime을 시작한다.

effect가 시작되고 관찰되는 경계

top-level import가 생기면 선택되지 않은 표면도 초기화 side effect를 만들 수 있다.

현재 설계가 선택한 trade-off와 비용

registry보다 명시적 early return을 택해 startup cost와 DCE 가능성을 우선한다. 이 선택의 계산·운영 비용은 인자 수가 작을 때 사실상 O(R), R은 route predicate 수다.로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

겹치는 predicate나 gate/policy 불일치는 잘못된 owner를 선택하거나 사용 불가능한 route를 노출한다. 이때 함께 대입할 반례는 predicate 순서 충돌, top-level import의 조기 side effect, 그리고 gate와 runtime policy의 불일치이다. 이들을 모두 하나의 "실패"로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

route name, 선택 predicate, startup checkpoint를 함께 기록해야 느린 초기화와 잘못된 dispatch를 구분할 수 있다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A03 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	고정 source에는 명시적 순서의 early return과 route별 dynamic import가 있다.
CONDITIONAL	compile-time feature 값과 argv 조합에 따라 일부 route만 후보가 된다.
UNKNOWN	배포 variant별 literal 값과 실제 사용 빈도는 snapshot으로 확정할 수 없다.
NEXT READ	새 route 추가 시 앞뒤 predicate의 교집합과 선택되지 않은 module의 top-level effect를 먼저 검사해야 한다.

다른 입력과 변경안에 전이해 볼 질문

1. 새 --remote-dev flag가 기존 remote predicate에도 맞는다면 어느 route가 선택되며 그 근거는 어디에 있는가?
2. dynamic import를 static import로 바꾸면 startup과 bundle variant에서 어떤 보장이 사라지는가?

이 설명을 확인할 코드 src/entrypoints/cli.tsx:28-93

SOURCE READING · 1/2

OBSERVED

Dynamic import가 architecture boundary 역할을 한다

import 시점은 성능뿐 아니라 side effect와 feature exclusion을 결정한다.

daemon/remote route의 lazy import

src/entrypoints/cli.tsx:95-180

```
95      // Fast-path for `--daemon-worker=<kind>` (internal — supervisor spawns this).
96      // Must come before the daemon subcommand check: spawned per-worker, so
97      // perf-sensitive. No enableConfigs(), no analytics sinks at this layer —
98      // workers are lean. If a worker kind needs configs/auth (assistant will),
99      // it calls them inside its run() fn.
100     if (feature('DAEMON') && args[0] === '--daemon-worker') {
101       const {
102         runDaemonWorker
103       } = await import('../daemon/workerRegistry.js');
104       await runDaemonWorker(args[1]);
105       return;
106     }
107
108     // Fast-path for `claude remote-control` (also accepts legacy `claude remote` / `claude sync` / `claude bridge`):
109     // serve local machine as bridge environment.
110     // feature() must stay inline for build-time dead code elimination;
111     // isBridgeEnabled() checks the runtime GrowthBook gate.
112     if (feature('BRIDGE_MODE') && (args[0] === 'remote-control' || args[0] === 'rc' || args[0] === 'remote' ||
args[0] === 'sync' || args[0] === 'bridge')) {
113       profileCheckpoint('cli_bridge_path');
114       const {
115         enableConfigs
116       } = await import('../utils/config.js');
117       enableConfigs();
118       const {
119         getBridgeDisabledReason,
… 56 lines omitted; use the source link for the complete range …
176         daemonMain
177       } = await import('../daemon/main.js');
178       await daemonMain(args.slice(1));
179       return;
180     }
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	route 판정 뒤 config를 enable한다.
그 표현이 만든 조건	auth와 policy를 route 안에서 읽는다.
그 표현이 만든 조건	누락 internal module은 false gate 뒤에 남는다.

언어와 런타임은 여기까지 보장한다

ESM dynamic import는 Promise로 module evaluation을 늦춘다. 하지만 import된 module의 top-level side effect는 그 시점에 실행된다.

프로젝트가 이 메커니즘에 맡긴 책임

entrypoint가 composition과 policy check 순서를 함께 소유한다.

경계를 잘못 읽을 때 생기는 위험

route별 초기화 순서가 달라져 shared invariant가 drift할 수 있다.

이 설명을 확인할 코드 src/entrypoints/cli.tsx:95-180

CONCEPT IN CONTEXT

INFERENCE

main.tsx는 composition root이면서 결합 압력점이다

환경, 설정, 인증, mode 선택, UI 준비가 한 장소에 모인다.

코드에서 읽어낼 수 있는 단서

출발 단서	main.tsx는 약 4.7천 줄이다.
연결되는 단서	변경 이유가 bootstrap, auth, onboarding, render, print dispatch까지 넓다.
연결되는 단서	파일 크기보다 lifetime이 다른 dependency가 함께 있다는 점이 핵심이다.

이 단서들을 하나의 설계로 묶으면

composition root는 원래 구체 dependency를 조립하는 곳이지만 product policy까지 축적되면 작은 기능이 startup 전반을 건드린다.

시스템 이해에 남는 결론

분해 시 순수 option resolution, external resource initialization, surface composition을 나누고 산출물을 명시해야 한다.

이 설명을 확인할 코드 src/main.tsx:240-310 src/main.tsx:3900-3940

SOURCE READING · 1/2

OBSERVED

launchRepl은 얇지만 REPL은 얇지 않다

launcher는 App과 REPL을 조립할 뿐 실제 application orchestration은 REPL 내부에 남는다.

React composition wrapper src/replLauncher.tsx:1-22

```
1     import React from 'react';
2     import type { StatsStore } from './context/stats.js';
3     import type { Root } from './ink.js';
4     import type { Props as REPLProps } from './screens/REPL.js';
5     import type { AppState } from './state/AppStateStore.js';
6     import type { FpsMetrics } from './utils/fpsTracker.js';
7     type AppWrapperProps = {
8       getFpsMetrics: () => FpsMetrics | undefined;
9       stats?: StatsStore;
10      initialState: AppState;
11    };
12    export async function launchRepl(root: Root, appProps: AppWrapperProps, replProps: REPLProps, renderAndRun: (root:
Root, element: React.ReactNode) => Promise<void>): Promise<void> {
13      const {
14        App
15      } = await import('./components/App.js');
16      const {
17        REPL
18      } = await import('./screens/REPL.js');
19      await renderAndRun(root, <App {...appProps}>
20          <REPL {...replProps} />
21        </App>);
22    }
```

> 이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	동적 import로 App과 REPL을 늦게 load한다.
그 표현이 만든 조건	initialState와 replProps를 별도 전달한다.
그 표현이 만든 조건	renderAndRun을 외부 port로 받는다.

언어와 런타임은 여기까지 보장한다

고차 함수 injection은 concrete renderer를 늦게 결합한다.

프로젝트가 이 메커니즘에 맡긴 책임

launcher의 얇음만 보고 UI adapter 전체가 얇다고 판단하면 안 된다.

경계를 잘못 읽을 때 생기는 위험

실제 query 호출과 state transition은 REPL.tsx에 있다.

이 설명을 확인할 코드 src/replLauncher.tsx:1-22

DISTINCTION TABLE

OBSERVED

실행 모드별 계약

같은 prompt라도 입력·권한·출력 소비자가 다르다.

표면	입력 owner	permission	출력
REPL	React event/controller	modal과 사용자 응답	Ink projection
print	stdin/command queue	non-interactive callback	structured stdout
SDK	host process	control callback/schema	SDKMessage stream
remote	network/bridge	transported decision	remote event projection

이 표를 읽는 관점

표면 차이는 view만이 아니라 control protocol의 차이다.

02 React와 Ink는 stream을 화면 상태로 어떻게 투영하는가

터미널 UI를 “React로 그린다”는 설명만으로는 부족하다. 현재 REPL은 render snapshot, 즉시 읽어야 하는 mutable state, query lifecycle, keyboard authority를 서로 다른 메커니즘으로 조정한다.

이 장의 논증은 다음 질문을 차례로 해결한다

코드	상태 소유 QueryGuard는 render state와 query lifecycle을 분리한다	코드	상태 소유 messagesRef와 React state는 같은 값의 서로 다른 시간 표현이다
코드	책임 연결 onQueryEvent는 event를 그대로 그리지 않고 보존 정책에 따라 축약한다	코드	진입 경계 useInput의 isActive와 propagation은 키보드 권한의 일부다
구분	진입 경계 REPL 안의 네 가지 상태 표현	해석	책임 연결 영상의 “React 앱” 직관은 출발점이고 현재 코드의 결론은 더 복잡하다

코드 독해의 시작점

`src/screens/REPL.tsx` → `src/utils/QueryGuard.ts` → `src/ink.ts`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

SOURCE READING · 1/2

OBSERVED

QueryGuard는 render state와 query lifecycle을 분리한다

local query의 실행 여부는 React의 비동기 state 두 개가 아니라 동기 상태 기계가 소유하고, React는 그 snapshot을 구독해 화면을 갱신한다.

QueryGuard와 useSyncExternalStore

src/screens/REPL.tsx:895-916

```
895     const lastUserScrollTsRef = useRef(0);
896
897     // Synchronous state machine for the query lifecycle. Replaces the
898     // error-prone dual-state pattern where isLoading (React state, async
899     // batched) and isQueryRunning (ref, sync) could desync. See QueryGuard.ts.
900     const queryGuard = React.useRef(new QueryGuard()).current;
901
902     // Subscribe to the guard — true during dispatching or running.
903     // This is the single source of truth for "is a local query in flight".
904     const isQueryActive = React.useSyncExternalStore(queryGuard.subscribe, queryGuard.getSnapshot);
905
906     // Separate loading flag for operations outside the local query guard:
907     // remote sessions (useRemoteSession / useDirectConnect) and foregrounded
908     // background tasks (useSessionBackgrounding). These don't route through
909     // onQuery / queryGuard, so they need their own spinner-visibility state.
910     // Initialize true if remote mode with initial prompt (CCR processing it).
911     const [isExternalLoading, setIsExternalLoadingRaw] = React.useState(remoteSessionConfig?.hasInitialPrompt ??
false);
912
913     // Derived: any loading source active. Read-only — no setter. Local query
914     // loading is driven by queryGuard (reserve/tryStart/end/cancelReservation),
915     // external loading by setIsExternalLoading.
916     const isLoading = isQueryActive || isExternalLoading;
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	QueryGuard instance는 useRef로 한 component lifetime 동안 유지된다.
그 표현이 만든 조건	useSyncExternalStore가 subscribe와 getSnapshot을 React render에 연결한다.
그 표현이 만든 조건	외부 session의 loading은 별도 state로 남고 최종 isLoading은 두 원천의 합성값이다.

언어와 런타임은 여기까지 보장한다

useSyncExternalStore는 React 바깥의 mutable store가 바뀔 때 snapshot을 다시 읽고 component render를 예약한다. 이 메커니즘은 store 자체를 React state로 바꾸지 않으며, snapshot의 일관성과 subscribe/unsubscribe 계약을 요구한다.

프로젝트가 이 메커니즘에 맡긴 책임

프로젝트는 “query가 실행 중인가”라는 control state를 QueryGuard의 동기 전이에 맡기고, spinner와 input disabled 같은 화면 판단만 React snapshot으로 투영한다. 이는 render scheduling이 실행 권한을 소유하지 않게 하는 선택이다.

경계를 잘못 읽을 때 생기는 위험

QueryGuard의 transition과 external loading의 setter가 서로 다른 lifetime을 갖는데 이를 하나의 boolean으로 되돌리면 순간적인 idle 표시, 중복 submit, 잘못된 cancel 대상이 다시 생길 수 있다.

이 설명을 확인할 코드 src/screens/REPL.tsx:895-916 src/utils/QueryGuard.ts:1-118

SOURCE READING · 1/2

OBSERVED

messagesRef와 React state는 같은 값의 서로 다른 시간 표현이다

stream callback이 바로 다음 문장에서 최신 message를 읽어야 하므로 REPL은 ref를 즉시 갱신하고 React state를 render projection으로 사용한다.

eager ref update와 render projection src/screens/REPL.tsx:1182-1212

```
1182    const [messages, rawSetMessages] = useState<MessageType[]>(initialMessages ?? []);
1183    const messagesRef = useRef(messages);
1184    // Stores the willowMode variant that was shown (or false if no hint shown).
1185    // Captured at hint_shown time so hint_converted telemetry reports the same
1186    // variant — the GrowthBook value shouldn't change mid-session, but reading
1187    // it once guarantees consistency between the paired events.
1188    const idleHintShownRef = useRef<string | false>(false);
1189    // Wrap setMessages so messagesRef is always current the instant the
1190    // call returns — not when React later processes the batch.  Apply the
1191    // updater eagerly against the ref, then hand React the computed value
1192    // (not the function).  rawSetMessages batching becomes last-write-wins,
1193    // and the last write is correct because each call composes against the
1194    // already-updated ref.  This is the Zustand pattern: ref is source of
1195    // truth, React state is the render projection.  Without this, paths
1196    // that queue functional updaters then synchronously read the ref
1197    // (e.g. handleSpeculationAccept → onQuery) see stale data.
1198    const setMessages = useCallback((action: React.SetStateAction<MessageType[]>) => {
1199      const prev = messagesRef.current;
… 8 lines omitted; use the source link for the complete range …
1208        // added messages don't include it (bridge status, hook results,
1209        // scheduled tasks landing async during processUserInputBase), bump
1210        // baseline so the placeholder stays visible. Once the user message
1211        // lands, stop tracking — later additions (assistant stream) should
1212        // not re-show the placeholder.
```

> 이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	messages는 화면을 다시 계산하게 하는 React state다.
그 표현이 만든 조건	messagesRef는 setter가 돌아오기 전에 다음 값을 가리킨다.
그 표현이 만든 조건	functional updater를 ref의 현재 값에 즉시 적용한 뒤 계산된 배열을 raw setter에 넘긴다.
그 표현이 만든 조건	compact·rewind와 pending user message는 같은 wrapper에서 baseline을 보정한다.

언어와 런타임은 여기까지 보장한다

React의 state 값은 현재 render의 snapshot이고 setter는 다음 render를 요청한다. 따라서 async callback이나 같은 tick의 후속 코드가 기존 closure의 state를 읽으면 stale할 수 있다. ref write는 render를 일으키지 않지만 같은 JavaScript 실행 안에서 즉시 관찰된다.

프로젝트가 이 메커니즘에 맡긴 책임

이 wrapper는 하나의 논리 state에 대해 ref를 control-side current value로, React state를 view-side projection으로 둔다. 마지막 write가 앞선 functional update를 잃지 않도록 모든 write가 wrapper를 통과해야 한다.

경계를 잘못 읽을 때 생기는 위험

rawSetMessages를 우회하거나 두 표현을 서로 다른 규칙으로 갱신하면 UI에는 보이지만 query context에는 빠진 message, 또는 그 반대가 생긴다. 이 패턴은 편의용 복제가 아니라 single-writer 규칙을 요구한다.

이 설명을 확인할 코드 src/screens/REPL.tsx:1182-1212

SOURCE READING · 1/2

OBSERVED

onQueryEvent는 event를 그대로 그리지 않고 보존 정책에 따라 축약한다

query가 yield한 message는 종류에 따라 append, replace, reset, tombstone removal로 투영되며 이 선택이 UI working set과 transcript 의미를 동시에 결정한다.

stream event의 UI projection

src/screens/REPL.tsx:2584-2660

```
2584    const onQueryEvent = useCallback((event: Parameters<typeof handleMessageFromStream>[0]) => {
2585      handleMessageFromStream(event, newMessage => {
2586        if (isCompactBoundaryMessage(newMessage)) {
2587          // Fullscreen: keep pre-compact messages for scrollback. query.ts
2588          // slices at the boundary for API calls, Messages.tsx skips the
2589          // boundary filter in fullscreen, and useLogMessages treats this
2590          // as an incremental append (first uuid unchanged). Cap at one
2591          // compact-interval of scrollback — normalizeMessages/applyGrouping
2592          // are O(n) per render, so drop everything before the previous
2593          // boundary to keep n bounded across multi-day sessions.
2594          if (isFullscreenEnvEnabled()) {
2595            setMessages(old => [...getMessagesAfterCompactBoundary(old, {
2596              includeSnipped: true
2597            }), newMessage]);
2598          } else {
2599            setMessages(() => [newMessage]);
2600          }
2601          // Bump conversationId so Messages.tsx row keys change and
2602          // stale memoized rows remount with post-compact content.
2603          setConversationId(randomUUID());
2604          // Compaction succeeded — clear the context-blocked flag so ticks resume
2605          if (feature('PROACTIVE') || feature('KAIROS')) {
2606            proactiveModule?.setContextBlocked(false);
2607          }

… 48 lines omitted; use the source link for the complete range …

2656          responseLengthBaseline: baseline,
2657          endResponseLength: baseline
2658        }));
2659      }, onStreamingText);
2660    }, [setMessages, setResponseLength, setStreamMode, setStreamingToolUses, setStreamingThinking, onStreamingText]);
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	compact boundary는 이전 interval만 남기고 새 boundary를 추가한 뒤 conversationId를 바꾼다.
그 표현이 만든 조건	ephemeral progress는 같은 tool call과 같은 progress type인 마지막 항목만 교체한다.
그 표현이 만든 조건	agent·hook·skill progress는 distinct history이므로 교체하지 않는다.
그 표현이 만든 조건	tombstone은 화면 배열과 transcript에서 함께 제거한다.

언어와 런타임은 여기까지 보장한다

functional state updater는 commit 시점의 이전 배열을 입력으로 받아 새 immutable 배열을 만든다. 같은 길이의 마지막 원소 교체는 append-only history와 다른 연산이며, memoized row identity와 persistence trigger에도 다른 신호를 준다.

프로젝트가 이 메커니즘에 맡긴 책임

REPL은 모든 stream event를 동일한 영구 message로 취급하지 않는다. 관찰 가치가 누적되는 event와 마지막 상태만 필요한 tick을 분류해 render 비용을 제한하고, compaction 때에는 key identity까지 무효화한다.

경계를 잘못 읽을 때 생기는 위험

ephemeral 분류를 넓히면 필요한 subagent 진행 이력이 사라지고, 좁히면 초당 tick이 수만 개 쌓인다. parentToolUseID나 data.type 비교가 틀리면 서로 다른 작업의 progress를 덮어쓸 수 있다.

이 설명을 확인할 코드 src/screens/REPL.tsx:2584-2660

SOURCE READING · 1/2

OBSERVED

useInput의 isActive와 propagation은 키보드 권한의 일부다

터미널에서는 여러 hook이 같은 stdin event를 볼 수 있으므로, 화면 상태로 handler를 gate하고 소비한 event를 명시적으로 차단해야 한다.

transcript search input gate src/screens/REPL.tsx:4201-4245

```
4201    // gates the useInput. Query persists across bar open/close so n/N keep
4202    // working after Enter dismisses the bar (less semantics).
4203    const jumpRef = useRef<JumpHandle | null>(null);
4204    const [searchOpen, setSearchOpen] = useState(false);
4205    const [searchQuery, setSearchQuery] = useState('');
4206    const [searchCount, setSearchCount] = useState(0);
4207    const [searchCurrent, setSearchCurrent] = useState(0);
4208    const onSearchMatchesChange = useCallback((count: number, current: number) => {
4209      setSearchCount(count);
4210      setSearchCurrent(current);
4211    }, []);
4212    useInput((input, key, event) => {
4213      if (key.ctrl || key.meta) return;
4214      // No Esc handling here — less has no navigating mode. Search state
4215      // (highlights, n/N) is just state. Esc/q/ctrl+c → transcript:exit
4216      // (ungated). Highlights clear on exit via the screen-change effect.
4217      if (input === '/') {
4218        // Capture scrollTop NOW — typing is a preview, 0-matches snaps
4219        // back here. Synchronous ref write, fires before the bar's
4220        // mount-effect calls setSearchQuery.
4221        jumpRef.current?.setAnchor();
4222        setSearchOpen(true);
4223        event.stopImmediatePropagation();
4224        return;
4225      }
… 15 lines omitted; use the source link for the complete range …
4241    const {
4242      setQuery: setHighlight,
4243      scanElement,
4244      setPositions
4245    } = useSearchHighlight();
```

> 이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	hook은 조건부로 호출되지 않고 항상 같은 위치에 선언된다.
그 표현이 만든 조건	isActive가 transcript·virtual scroll·search dialog 상태에 따라 handler를 켠다.
그 표현이 만든 조건	검색 시작과 n/N 탐색이 event.stopImmediatePropagation으로 뒤 handler를 막는다.
그 표현이 만든 조건	paste처럼 여러 문자가 한 event에 올 수 있어 동일 문자 batch를 반복 횟수로 해석한다.

언어와 런타임은 여기까지 보장한다

React hook은 render마다 같은 호출 순서를 유지해야 하므로 화면 분기 안에서 hook 자체를 만들고 없애는 대신 option으로 구독의 활성도를 바꾼다. Ink의 useInput은 stdin을 event로 정규화하지만, 여러 handler 사이의 우선순위와 소비 여부는 application이 설계해야 한다.

프로젝트가 이 메커니즘에 맡긴 책임

이 코드는 키 입력을 단순 callback이 아니라 현재 modal·screen이 소유하는 capability로 취급한다. transcript search가 활성일 때 `/`, `n`, `N`의 의미는 prompt editor나 global shortcut으로 전파되지 않아야 한다.

경계를 잘못 읽을 때 생기는 위험

gate 조건과 propagation stop이 어긋나면 하나의 키가 검색 이동과 prompt 편집을 동시에 실행하거나, 비활성 화면의 handler가 전역 shortcut을 가로챌 수 있다.

이 설명을 확인할 코드 src/screens/REPL.tsx:4201-4245

DISTINCTION TABLE

OBSERVED

REPL 안의 네 가지 상태 표현

React를 쓴다는 사실보다 중요한 것은 값마다 필요한 관찰 시점과 owner에 맞는 표현을 골랐다는 점이다.

표현	읽히는 시점	render 유발	대표 책임
React state	현재 render snapshot	setter가 다음 render 예약	messages·dialog·입력 표시
ref	같은 tick의 즉시 mutable read	아니오	messagesRef·abort target·시간 측정
external store	subscribe 후 snapshot	store 변화가 render 연결	QueryGuard lifecycle
stream event	async iterator가 전달할 때	직접 아님	onQueryEvent가 projection 선택

이 표를 읽는 관점

네 표현은 서로 대체 가능한 취향이 아니다. 실행 권한, 최신 값, 화면 재계산, 외부 protocol이라는 서로 다른 시간 계약을 맡는다.

CONCEPT IN CONTEXT

INFERENCE

영상의 "React 앱" 직관은 출발점이고 현재 코드의 결론은 더 복잡하다

React reconciler와 Ink renderer가 터미널 tree를 갱신한다는 설명은 유효하지만, UI layer가 나머지와 깔끔히 분리되었다고 일반화하면 REPL이 가진 orchestration 책임을 놓친다.

코드에서 읽어낼 수 있는 단서

출발 단서	JSX와 hook은 DOM 대신 Ink의 terminal renderer로 투영된다.
연결되는 단서	query event는 React가 생산하지 않고 async generator에서 들어온다.
연결되는 단서	REPL은 그 event를 화면에 그리는 동시에 message working set, compaction identity, query lifecycle, permission interaction을 조정한다.
연결되는 단서	따라서 "React UI"와 "application controller"가 현재 같은 큰 component 안에 공존한다.

이 단서들을 하나의 설계로 묶으면

영상은 React/renderer 분리와 declarative streaming의 학습 직관을 준다. 그러나 이 snapshot에서는 단순한 setStreamingText 예보다 QueryGuard, eager ref update, event 종류별 replace/append, input propagation이 실제 보장을 만든다. 외부 설명은 이 코드를 읽을 질문을 제공할 뿐 upstream 의도를 증명하지 않는다.

시스템 이해에 남는 결론

리팩터링 목표도 React를 제거하거나 모든 state를 store 하나에 넣는 것이 아니다. query·permission·transcript의 canonical transition은 application boundary로 옮기고, Ink component에는 render snapshot과 interaction projection을 남기는 쪽이 현재 결합을 더 정확히 푼다.

언어·학습 참고자료 영상: Claude Code 까보니 React 앱이었습니다 · 영상 내용을 정리한 PEC 분석 글 · React 공식 문서: useSyncExternalStore · React 공식 문서: state update queue와 batching · Ink 공식 저장소: useInput과 terminal renderer

이 설명을 확인할 코드 src/screens/REPL.tsx:2584-2660 src/screens/REPL.tsx:2788-2804

03 React 아래에는 별도의 terminal frame runtime이 있다

JSX가 바뀐 뒤 화면에 글자가 보이기까지는 Yoga layout, cell buffer, damage 계산, frame diff, ANSI write가 이어진다. 이 층을 생략하면 streaming UI의 성능과 correctness를 React state만으로 설명하게 된다.

이 장의 논증은 다음 질문을 차례로 해결한다

코드	책임 연결 Custom Ink renderer는 Yoga tree를 reusable screen buffer로 투영한다
구분	책임 연결 한 frame의 네 표현은 서로 다른 보증을 맡는다
코드	책임 연결 damage와 double buffer가 물리 화면 commit 범위를 결정한다
사례	책임 연결 streaming token 하나가 화면에 보일 때

코드 독해의 시작점

`src/ink/renderer.ts` → `src/ink/output.ts` → `src/ink/ink.tsx` → `src/ink/terminal.ts`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

SOURCE READING · 1/2

OBSERVED

Custom Ink renderer는 Yoga tree를 reusable screen buffer로 투영한다

현재 renderer는 React node를 문자열로 곧장 출력하지 않고 layout이 계산된 tree를 cell 기반 back screen에 그린다.

layout 검증, back buffer reset, screen render

src/ink/renderer.ts:35-149

```
35      // Reuse Output across frames so charCache (tokenize + grapheme clustering)
36      // persists — most lines don't change between renders.
37      let output: Output | undefined
38      return options => {
39        const { frontFrame, backFrame, isTTY, terminalWidth, terminalRows } =
40          options
41        const prevScreen = frontFrame.screen
42        const backScreen = backFrame.screen
43        // Read pools from the back buffer's screen — pools may be replaced
44        // between frames (generational reset), so we can't capture them in the closure
45        const charPool = backScreen.charPool
46        const hyperlinkPool = backScreen.hyperlinkPool
47
48        // Return empty frame if yoga node doesn't exist or layout hasn't been computed yet.
49        // getComputedHeight() returns NaN before calculateLayout() is called.
50        // Also check for invalid dimensions (negative, Infinity) that would cause RangeError
51        // when creating arrays.
52        const computedHeight = node.yogaNode?.getComputedHeight()
53        const computedWidth = node.yogaNode?.getComputedWidth()
54        const hasInvalidHeight =
55          computedHeight === undefined ||
56          !Number.isFinite(computedHeight) ||
57          computedHeight < 0
58        const hasInvalidWidth =
59          computedWidth === undefined ||
… 85 lines omitted; use the source link for the complete range …
145
146       return {
147         scrollHint: options.altScreen ? getScrollHint() : null,
148         scrollDrainPending: drainNode !== null,
149         screen: renderedScreen,
```

> 이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	Yoga의 computed width/height가 없거나 유한하지 않으면 높이 0의 안전한 frame을 돌려준다.
그 표현이 만든 조건	alternate screen에서는 physical terminal rows를 screen height의 상한으로 강제한다.
그 표현이 만든 조건	Output object와 char cache를 frame 사이에 재사용한다.
그 표현이 만든 조건	이전 frame이 selection overlay나 absolute removal로 오염됐으면 blit fast path를 포기한다.

언어와 런타임은 여기까지 보장한다

Yoga는 flex layout을 계산하지만 실제 terminal cell과 ANSI escape sequence를 알지 못한다. Output은 layout 결과를 grapheme·width·style을 가진 screen cell로 rasterize한다. front/back buffer는 “React render가 끝났다”와 “terminal에 새 frame이 commit됐다” 사이의 중간 표현이다.

프로젝트가 이 메커니즘에 맡긴 책임

이 구조는 고정 높이 streaming에서 변경되지 않은 subtree를 blit해 다시 grapheme clustering하는 비용을 줄이는 대신, 이전 screen이 신뢰 가능한지를 명시적으로 추적해야 한다.

경계를 잘못 읽을 때 생기는 위험

오염된 screen을 재사용하거나 alt-screen 높이 불변조건을 깨면 stale cell, cursor drift, 마지막 행 scroll처럼 state 값은 맞지만 물리 화면이 틀린 오류가 생긴다.

이 설명을 확인할 코드 src/ink/renderer.ts:35-149 src/ink/output.ts:170-225

SOURCE READING · 1/2

OBSERVED

damage와 double buffer가 물리 화면 commit 범위를 결정한다

새 frame 전체를 매번 쓰지 않고 바뀐 cell만 출력하려면 어떤 region이 달라졌는지와 이전 frame의 신뢰도를 함께 보존해야 한다.

overlay, full damage backstop, frame diff와 swap

src/ink/ink.tsx:515-595

```
515       // Selection overlay: invert cell styles in the screen buffer itself,
516       // so the diff picks up selection as ordinary cell changes and
517       // LogUpdate remains a pure diff engine.
518       //
519       // Full-screen damage (PR #20120) is a correctness backstop for the
520       // sibling-resize bleed: when flexbox siblings resize between frames
521       // (spinner appears → bottom grows → scrollbox shrinks), the
522       // cached-clear + clip-and-cull + setCellAt damage union can miss
523       // transition cells at the boundary. But that only happens when layout
524       // actually SHIFTS — didLayoutShift() tracks exactly this (any node's
525       // cached yoga position/size differs from current, or a child was
526       // removed). Steady-state frames (spinner rotate, clock tick, text
527       // stream into fixed-height box) don't shift layout, so normal damage
528       // bounds are correct and diffEach only compares the damaged region.
529       //
530       // Selection also requires full damage: overlay writes via setCellStyleId
531       // which doesn't track damage, and prev-frame overlay cells need to be
532       // compared when selection moves/clears. prevFrameContaminated covers
533       // the frame-after-selection-clears case.
534       let selActive = false;
535       let hlActive = false;
536       if (this.altScreenActive) {
… 54 lines omitted; use the source link for the complete range …
591       SYNC_OUTPUT_SUPPORTED);
592       const diffMs = performance.now() - tDiff;
593       // Swap buffers
594       this.backFrame = this.frontFrame;
595       this.frontFrame = frame;
```

> 이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	selection과 search highlight는 screen style을 직접 바꾸므로 full damage가 필요하다.
그 표현이 만든 조건	layout shift는 sibling 경계의 stale cell을 막기 위해 full-screen damage로 승격된다.
그 표현이 만든 조건	alt-screen은 diff 전에 physical cursor를 (0,0) 기준으로 다시 고정한다.
그 표현이 만든 조건	diff를 계산한 뒤 back/front frame을 swap하고, 최적화된 patch를 terminal에 쓴다.

언어와 런타임은 여기까지 보장한다

double buffering은 다음 frame을 별도 buffer에 만든 뒤 이전 frame과 비교하는 표현이다. damage rectangle은 비교해야 할 후보를 줄이는 최적화일 뿐, 잘못 좁히면 correctness가 깨진다. 그래서 layout shift·overlay·contamination은 더 비싼 full damage를 선택하는 보수적 fallback이다.

프로젝트가 이 메커니즘에 맡긴 책임

화면의 canonical React state와 physical terminal은 동시에 바뀌지 않는다. screen diff write가 commit 경계이며, 그 뒤에만 새 front frame을 다음 비교 기준으로 신뢰할 수 있다.

경계를 잘못 읽을 때 생기는 위험

tmux나 emulator가 cursor를 바깥에서 움직인 경우 relative diff만으로는 drift가 누적된다. main screen과 alt screen의 cursor·scrollback semantics가 달라 같은 reset을 무조건 적용할 수도 없다.

이 설명을 확인할 코드 src/ink/ink.tsx:515-595 src/ink/ink.tsx:720-742

DISTINCTION TABLE

한 frame의 네 표현은 서로 다른 보증을 맡는다

OBSERVED

"React가 터미널을 그린다"를 실제 변경 경계로 펼치면 각 표현의 owner와 실패를 분리할 수 있다.

표현	소유하는 사실	다음 경계에서 필요한 보증
React tree	어떤 component와 props가 존재하는가	reconciler가 host node mutation을 일관되게 반영
Yoga layout	각 node의 x·y·width·height	유한한 dimension과 alt-screen viewport 불변조건
Screen cells	grapheme·width·style·hyperlink의 frame snapshot	wide character와 overlay를 cell 단위로 보존
Terminal patches	ANSI 이동·지우기·쓰기의 최소 sequence	이전 physical cursor/frame 가정과 write 원자성

이 표를 읽는 관점

위 표현 중 하나를 건너뛰면 "state는 맞는데 화면이 깨짐", "화면은 맞는데 선택 text가 틀림", "빠른 frame에서만 stale line" 같은 오류의 owner를 찾을 수 없다.

SELECTIVE EXECUTION TRACE

OBSERVED

streaming token 하나가 화면에 보일 때

실행 순서를 나열하려는 목적이 아니라 각 단계에서 어떤 representation이 새 truth를 얻는지 확인한다.

관찰·정산

Projection

onQueryEvent가 partial message의 view state를 갱신하고 React render를 예약한다.

이때 남은 상태와 판단이 “Reconciliation”에서 사용할 전제가 된다.

책임 연결

Reconciliation

host node의 text와 dirty state가 바뀌며 Yoga가 필요한 layout을 계산한다.

이때 남은 상태와 판단이 “Rasterization”에서 사용할 전제가 된다.

책임 연결

Rasterization

Output이 back screen의 cell을 쓰고 unchanged subtree는 안전할 때 이전 screen에서 blit한다.

이때 남은 상태와 판단이 “Assurance”에서 사용할 전제가 된다.

책임 연결

Assurance

layout shift·selection·contamination이면 damage를 전체 screen으로 넓혀 stale cell을 피한다.

이때 남은 상태와 판단이 “Commit”에서 사용할 전제가 된다.

책임 연결

Commit

front/back diff를 ANSI patch로 바꾸어 terminal에 쓰고 새 frame을 다음 비교 기준으로 삼는다.

이 지점에서 앞선 상태 변화가 외부에서 관찰 가능한 결과로 정산된다.

이 흐름 전체에서 보존해야 하는 것

partial message의 배열 갱신은 화면 commit이 아니다. terminal write가 끝나기 전에는 application state와 physical screen이 서로 다른 시점의 projection일 수 있다.

어디에서 설명이 틀어지는가

process가 write 중 종료되거나 external program이 cursor를 움직이면 transcript truth는 남아도 physical frame은 복구가 필요하다.

이 설명을 확인할 코드 src/ink/renderer.ts:118-174 src/ink/ink.tsx:568-595

04 두 orchestration shell

현재 구현의 핵심은 REPL과 headless가 query() 위에서 서로 다른 state owner를 가진다는 점이다.

이 장의 논증은 다음 질문을 차례로 해결한다

코드	진입 경계 REPL은 query()를 직접 호출한다	코드	상태 소유 QueryEngine은 headless용 session abstraction이다
알고리즘	책임 연결 ask()의 one-shot engine adapter	알고리즘	책임 연결 print loop의 turn 간 상태 handoff
구분	상태 소유 실제 state owner 표		

코드 독해의 시작점

`src/screens/REPL.tsx` → `src/QueryEngine.ts` → `src/cli/print.ts`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

SOURCE READING · 1/2

OBSERVED

REPL은 query()를 직접 호출한다

REPL은 단순 renderer가 아니라 system prompt와 ToolUseContext를 구성하는 application controller다.

interactive query invocation

src/screens/REPL.tsx:2767-2804

```
2767      queryCheckpoint('query_context_loading_start');
2768      const [,, defaultSystemPrompt, baseUserContext, systemContext] = await Promise.all([
2769      // IMPORTANT: do this after setMessages() above, to avoid UI jank
2770      checkAndDisableBypassPermissionsIfNeeded(toolPermissionContext, setAppState),
2771      // Gated on TRANSCRIPT_CLASSIFIER so GrowthBook kill switch runs wherever auto mode is built in
2772      feature('TRANSCRIPT_CLASSIFIER') ? checkAndDisableAutoModeIfNeeded(toolPermissionContext, setAppState,
store.getState().fastMode) : undefined, getSystemPrompt(freshTools, mainLoopModelParam,
Array.from(toolPermissionContext.additionalWorkingDirectories.keys()), freshMcpClients), getUserContext(),
getSystemContext()]);
2773      const userContext = {
2774        ...baseUserContext,
2775        ...getCoordinatorUserContext(freshMcpClients, isScratchpadEnabled() ? getScratchpadDir() : undefined),
2776        ...((feature('PROACTIVE') || feature('KAIROS')) && proactiveModule?.isProactiveActive() &&
!terminalFocusRef.current ? {
2777          terminalFocus: 'The terminal is unfocused \u2014 the user is not actively watching.'
2778        } : {})
2779      };
2780      queryCheckpoint('query_context_loading_end');
2781      const systemPrompt = buildEffectiveSystemPrompt({
2782        mainThreadAgentDefinition,
2783        toolUseContext,
2784        customSystemPrompt,
2785        defaultSystemPrompt,
2786        appendSystemPrompt
2787      });
2788      toolUseContext.renderedSystemPrompt = systemPrompt;
2789      queryCheckpoint('query_query_start');
2790      resetTurnHookDuration();
… 9 lines omitted; use the source link for the complete range …
2800        querySource: getQuerySourceForREPL()
2801      })) {
2802        onQueryEvent(event);
2803      }
2804      if (feature('BUDDY')) {
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	permission mode를 먼저 점검한다.
그 표현이 만든 조건	system/user/system context를 조립한다.
그 표현이 만든 조건	ToolUseContext에 rendered prompt를 넣는다.
그 표현이 만든 조건	query event를 onQueryEvent로 직접 소비한다.

언어와 런타임은 여기까지 보장한다

React callback 안의 비동기 orchestration은 render와 effect를 분리하지만 application logic을 UI module에 결합한다.

프로젝트가 이 메커니즘에 맡긴 책임

현재 snapshot에서 QueryEngine은 이 경로의 owner가 아니다.

경계를 잘못 읽을 때 생기는 위험

SDK에 기능을 추가해도 REPL 경로에 자동 반영되지 않을 수 있다.

이 설명을 확인할 코드 src/screens/REPL.tsx:2767-2804

SOURCE READING · 1/2

OBSERVED

QueryEngine은 headless 용 session abstraction이다

class field는 submit 호출보다 오래 살아야 하는 상태를 모은다.

QueryEngine state ownership

src/QueryEngine.ts:175-213

```
175   /**
176    * QueryEngine owns the query lifecycle and session state for a conversation.
177    * It extracts the core logic from ask() into a standalone class that can be
178    * used by both the headless/SDK path and (in a future phase) the REPL.
179    *
180    * One QueryEngine per conversation. Each submitMessage() call starts a new
181    * turn within the same conversation. State (messages, file cache, usage, etc.)
182    * persists across turns.
183    */
184   export class QueryEngine {
185     private config: QueryEngineConfig
186     private mutableMessages: Message[]
187     private abortController: AbortController
188     private permissionDenials: SDKPermissionDenial[]
189     private totalUsage: NonNullableUsage
190     private hasHandledOrphanedPermission = false
191     private readFileState: FileStateCache
192     // Turn-scoped skill discovery tracking (feeds was_discovered on
193     // tengu_skill_tool_invocation). Must persist across the two
194     // processUserInputContext rebuilds inside submitMessage, but is cleared
195     // at the start of each submitMessage to avoid unbounded growth across
196     // many turns in SDK mode.
197     private discoveredSkillNames = new Set<string>()
198     private loadedNestedMemoryPaths = new Set<string>()
199
… 9 lines omitted; use the source link for the complete range …
209     async *submitMessage(
210       prompt: string | ContentBlockParam[],
211       options?: { uuid?: string; isMeta?: boolean },
212     ): AsyncGenerator<SDKMessage, void, unknown> {
213       const {
```

> 이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	mutableMessages
그 표현이 만든 조건	AbortController
그 표현이 만든 조건	permission denials
그 표현이 만든 조건	usage
그 표현이 만든 조건	readFileState
그 표현이 만든 조건	skill/memory dedup set

언어와 런타임은 여기까지 보장한다

class private state는 lifetime boundary를 표현하지만 process-wide singleton을 자동으로 제거하지는 않는다.

프로젝트가 이 메커니즘에 맡긴 책임

주석은 REPL 통합을 future phase로 명시한다.

경계를 잘못 읽을 때 생기는 위험

보고서가 이를 universal engine으로 표현하면 현재와 목표가 뒤섞인다.

이 설명을 확인할 코드 src/QueryEngine.ts:175-213

ALGORITHM A04 · 독자의 질문

ask()의 one-shot engine adapter은 어떤 시스템 문제를 닫는가

기존 headless caller는 함수 한 번으로 질문을 끝내고 싶지만 새 QueryEngine은 lifetime과 cleanup을 가진 객체다. adapter는 두 모델 사이에서 상태를 복사하고 종료를 정산해야 한다.

코드가 필요해지는 이유

기존 headless call site가 class lifetime을 직접 관리하지 않고 한 prompt를 실행하게 한다.

OBSERVED — ask()가 새 QueryEngine instance와 clone된 read cache를 call lifetime 동안 소유하고, 외부 mutable state로 결과를 돌려준다.

INFERENCE — 이 함수는 지속 session engine 자체가 아니라 기존 함수형 call site를 위한 compatibility seam이다.

이 절에서 답할 결정 질문

one-shot wrapper가 편의 API를 유지하면서도 generator close와 cache handoff를 잃지 않으려면 무엇을 소유해야 하는가?

이 설명을 확인할 코드 src/QueryEngine.ts:1249-1295

ALGORITHM A04 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/QueryEngine.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01
adapter와 resource lifetime

adapter는 이름만 바꾸는 wrapper가 아니라 서로 다른 상태·종료 계약을 번역한다. async generator는 consumer가 끝까지 소비하거나 close해야 finally가 실행된다.

> **실제 코드로 돌아가면.** 1249–1295행에서 instance 생성, iteration, finally cleanup, cache copy-back의 순서를 함께 확인한다.

이론은 정답이 아니라 코드에 던질 질문이다

구성·변형·성능
`adapter · state handoff`

조립 방식이나 최적화를 바꿔도 핵심 계약이 adapter와 build 변형 안에서 보존되는가?

검증·관측·운영
`resource cleanup via finally`

최종 출력뿐 아니라 실패·중단·복구 경로의 보장을 어떤 관찰과 테스트로 입증할 수 있는가?

ALGORITHM A04 · 실제 코드

OBSERVED

ask()의 one-shot engine adapter 의 결정점은 어디에 있는가

호환 wrapper는 외부 mutable state를 새 QueryEngine에 주입하고 결과 state를 다시 꺼낸다.

입력 표현. mutableMessages, read-file cache getter/setter, 그리고 prompt와 config

보존되는 작업 상태. 새 QueryEngine instance, 그리고 clone된 read cache

one-shot QueryEngine lifecycle

src/QueryEngine.ts:1249-1295

```
1249    const engine = new QueryEngine({
1250      cwd,
1251      tools,
1252      commands,
1253      mcpClients,
1254      agents,
1255      canUseTool,
1256      getAppState,
1257      setAppState,
1258      initialMessages: mutableMessages,
1259      readFileCache: cloneFileStateCache(getReadFileCache()),
1260      customSystemPrompt,
1261      appendSystemPrompt,
1262      userSpecifiedModel,
… 30 lines omitted; use the source link for the complete range …
1293      setReadFileCache(engine.getReadFileState())
1294    }
1295  }
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A04 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

mutableMessages, read-file cache getter/setter, 그리고 prompt와 config은 처음부터 신뢰된 실행 상태가 아니다. 새 QueryEngine instance, 그리고 clone된 read cache가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** 외부 상태로 engine을 생성한다. 이 결과가 다음 판단 "submitMessage를 yield*로 전달한다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** submitMessage를 yield*로 전달한다. 이 결과가 다음 판단 "정상·중단 종료와 무관하게 finally에서 file state를 되돌린다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** 정상·중단 종료와 무관하게 finally에서 file state를 되돌린다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

한 ask 호출에는 한 engine instance가 있다. message 배열은 caller와 합의한 방식으로 갱신된다. file state handoff는 generator 종료 시도에도 실행된다.

ALGORITHM A04 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **구성·변형·성능의 질문으로 읽기.** 한 ask 호출에는 한 engine instance가 있다.
- **검증·관측·운영의 질문으로 읽기.** message 배열은 caller와 합의한 방식으로 갱신된다.
- **구성·변형·성능의 질문으로 읽기.** file state handoff는 generator 종료 시도에도 실행된다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- consumer가 generator를 닫지 않아 finally가 늦어짐
- 외부 mutableMessages와 engine store의 drift
- turn 간 class-local field 소실

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 기존 함수 API를 보존하면서 class 기반 engine을 도입하는 compatibility seam이다.

RECOMMENDATION 지속 session consumer는 engine을 직접 소유하고, one-shot wrapper는 명시적으로 이름과 보증을 제한한다.

ALGORITHM A04 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

한 prompt마다 engine을 만들고 event를 소비한 뒤 cleanup과 외부 state handoff를 수행한다.

effect가 시작되고 관찰되는 경계

generator가 중간 종료되면 finally가 자원 정산의 마지막 경계가 된다.

현재 설계가 선택한 trade-off와 비용

기존 함수 API를 보존하면서 class 기반 engine을 도입하는 compatibility seam이다. 이 선택의 계산·운영 비용은 message/state 크기에 따른 clone 비용 + query 실행 비용.로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

consumer가 iterator를 닫지 않거나 새 session field가 copy-back 목록에 빠지면 완료된 응답과 지속 상태가 어긋난다. 이때 함께 대입할 반례는 consumer가 generator를 닫지 않아 finally가 늦어짐, 외부 mutableMessages와 engine store의 drift, 그리고 turn 간 class-local field 소실이다. 이들을 모두 하나의 "실패"로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

engine 생성·terminal reason·finally 진입·handoff field를 같은 call identity로 기록해야 한다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A04 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	one-shot instance 생성과 함수 API 호환 경로가 존재한다.
CONDITIONAL	caller가 iterator lifecycle을 정상적으로 닫을 때 cleanup과 state handoff가 완료된다.
UNKNOWN	모든 headless caller가 중간 중단 때 return()을 호출하는지는 이 범위에서 확인할 수 없다.
NEXT READ	지속 session consumer는 wrapper를 반복 호출하기보다 engine 직접 소유가 필요한지 call site별로 재검토해야 한다.

다른 입력과 변경안에 전이해 볼 질문

1. 두 turn 사이에 보존돼야 할 새 field가 생기면 adapter와 direct owner 중 어디를 바꿔야 하는가?
2. consumer가 첫 event 뒤 중단하면 finally와 read cache handoff가 실제로 실행되는가?

이 설명을 확인할 코드 src/QueryEngine.ts:1249-1295

ALGORITHM A05 · 독자의 질문

print loop의 turn 간 상태 handoff 은 어떤 시스템 문제를 닫는가

print loop는 여러 command를 연속 처리하지만 각 ask 호출은 새 engine을 만든다. 대화의 연속성은 engine 내부가 아니라 바깥 queue와 AppState의 handoff에 달려 있다.

코드가 필요해지는 이유

연속 stdin/command 입력을 독립 ask 호출로 처리하면서 conversation continuity를 유지한다.

OBSERVED — src/cli/print.ts의 command loop가 queue, mutableMessages, readFileState와 turn별 abort controller를 소유한다.

INFERENCE — QueryEngine은 per-call 실행자이고 print loop가 headless session의 장기 owner다. REPL과 같은 UI라도 ownership은 같다고 가정할 수 없다.

이 절에서 답할 결정 질문

clone-then-replace 경계에서 어떤 state가 turn을 건너고, concurrent input이 끼면 어느 값이 유실될 수 있는가?

이 설명을 확인할 코드 src/cli/print.ts:2140-2188

ALGORITHM A05 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/cli/print.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01

외부화된 상태기계와 copy-merge

함수가 매번 새로 만들어져도 caller가 이전 출력 상태를 다음 입력에 넣으면 하나의 장기 상태기계처럼 동작한다. 하지만 복사 후 전체 교체는 그 사이의 concurrent write를 덮을 수 있다.

> **실제 코드로 돌아가면.** 2140–2188행에서 ask에 넘긴 seed와 종료 뒤 되돌려 쓰는 state를 대조한다.

이론은 정답이 아니라 코드에 던질 질문이다

상태·계산 모델
`externalized state machine`
단계의 순서나 표현을 바꿔도 같은 상태와 불변조건에 도달하는가?

동시성·시간·합의
`versioned state`
stale read, 중복 실행, 취소가 끼어들어도 effect와 최종 상태의 의미가 보존되는가?

내구성·복구
`copy-merge`
중단 지점에서 재개했을 때 무엇을 재생하고 무엇을 다시 실행하지 않아야 하는가?

ALGORITHM A05 · 실제 코드

OBSERVED

print loop의 turn 간 상태 handoff 의 결정점은 어디에 있는가

print는 engine을 유지하지 않고 mutableMessages와 readFileState를 loop 바깥에서 보존한다.

입력 표현. dequeued command, 현재 tool/MCP 집합, mutableMessages, 그리고 readFileState

보존되는 작업 상태. loop-local abort controller, pending read seeds, 그리고 외부 AppState

headless ask invocation

src/cli/print.ts:2140-2188

```
2140            // Per-iteration ALS context so bg agents spawned inside ask()
2141            // inherit workload across their detached awaits. In-process cron
2142            // stamps cmd.workload; the SDK --workload flag is options.workload.
2143            // const-capture: TS loses `while ((command = dequeue()))` narrowing
2144            // inside the closure.
2145            const cmd = command
2146            await runWithWorkload(cmd.workload ?? options.workload, async () => {
2147              for await (const message of ask({
2148                commands: uniqBy(
2149                  [...currentCommands, ...appState.mcp.commands],
2150                  'name',
2151                ),
2152                prompt: input,
2153                promptUuid: cmd.uuid,
… 32 lines omitted; use the source link for the complete range …
2186                abortController,
2187                replayUserMessages: options.replayUserMessages,
2188                includePartialMessages: options.includePartialMessages,
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A05 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

dequeued command, 현재 tool/MCP 집합, mutableMessages, 그리고 readFileState은 처음부터 신뢰된 실행 상태가 아니다. loop-local abort controller, pending read seeds, 그리고 외부 AppState가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** command마다 AbortController를 만든다. 이 결과가 다음 판단 "ask()에 현재 history와 ports를 전달한다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** ask()에 현재 history와 ports를 전달한다. 이 결과가 다음 판단 "stream을 소비한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** stream을 소비한다. 이 결과가 다음 판단 "engine이 반환한 read cache와 pending seed를 병합한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** engine이 반환한 read cache와 pending seed를 병합한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

새 turn은 이전 message history를 본다. edit precondition용 read state가 turn 사이에 유지된다. pending seed가 더 최신이면 덮어쓴다.

ALGORITHM A05 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **상태·계산 모델의 질문으로 읽기.** 새 turn은 이전 message history를 본다.
- **내구성·복구의 질문으로 읽기.** edit precondition용 read state가 turn 사이에 유지된다.
- **동시성·시간·합의의 질문으로 읽기.** pending seed가 더 최신이면 덮어쓴다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- concurrent stdin seed와 engine clone-then-replace race
- 새 session field가 handoff 목록에서 누락

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED command queue owner가 session continuity를 갖고 QueryEngine은 per-call adapter가 된다.

RECOMMENDATION session state bundle을 단일 versioned object로 만들고 REPL/headless가 같은 reducer를 사용한다.

ALGORITHM A05 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

queue가 command를 꺼내고 현재 session bundle로 ask를 실행한 뒤 결과를 다음 turn의 seed로 저장한다.

effect가 시작되고 관찰되는 경계

stdin이나 callback이 같은 AppState를 갱신하면 engine 결과와 merge 순서가 의미를 바꾼다.

현재 설계가 선택한 trade-off와 비용

command queue owner가 session continuity를 갖고 QueryEngine은 per-call adapter가 된다. 이 선택의 계산·운영 비용은 각 turn은 query 비용, cache merge는 O(P) pending seeds.로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

새 session field의 누락 또는 clone 이후 외부 write는 다음 turn의 world model을 오래된 상태로 되돌릴 수 있다. 이때 함께 대입할 반례는 concurrent stdin seed와 engine clone-then-replace race, 그리고 새 session field가 handoff 목록에서 누락이다. 이들을 모두 하나의 "실패"로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

turn id, input dequeue 시각, seed version, returned version을 남기면 lost update를 찾을 수 있다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A05 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	print loop가 mutableMessages와 readFileState를 ask 호출 사이에 전달한다.
CONDITIONAL	동시에 같은 state를 쓰는 경로가 없고 handoff 목록이 완전할 때 continuity가 유지된다.
UNKNOWN	stdin seed와 engine completion이 실제 배포에서 얼마나 경쟁하는지는 확인되지 않는다.
NEXT READ	state bundle을 versioned object로 묶기 전 먼저 각 field의 writer와 lifetime을 표로 재검증해야 한다.

다른 입력과 변경안에 전이해 볼 질문

1. engine 실행 중 새 stdin message가 추가되면 종료 시 replace가 그 message를 보존하는가?
2. REPL과 print가 같은 reducer를 쓴다면 surface별로 남겨야 할 owner 차이는 무엇인가?

이 설명을 확인할 코드 src/cli/print.ts:2140-2188

DISTINCTION TABLE

OBSERVED

실제 state owner 표

이 표가 보고서 전체의 canonical ownership map이다.

상태	REPL	Headless/print	공유 여부
messages	React/ref/store	mutableMessages 외부 배열	표현은 공유, owner는 다름
read-file state	ToolUseContext/AppState 경로	print가 turn 간 보존	semantics 공유
abort	REPL state/controller	turn별 controller	protocol 공유
usage/denials	event/state projection	QueryEngine field	완전 동일하지 않음
UI modal	REPL only	host callback/event	공유하지 않음

이 표를 읽는 관점

새 기능은 먼저 어느 lifetime과 owner에 속하는지 결정해야 한다.

05 공통화할 것과 남겨둘 차이

좋은 개선은 UI를 engine에 밀어 넣는 것이 아니라 canonical event와 state transition만 공유한다.

이 장의 논증은 다음 질문을 차례로 해결한다

코드	상태 소유 ToolUseContext는 port와 mutable projection이 함께 있다	알고리즘	상태 소유 state lifetime classification
사례	책임 연결 새 progress event의 영향 분석	구분	책임 연결 공통화 대상과 surface 고유 책임
해석	진입 경계 권장 seam: ConversationRuntime		

코드 독해의 시작점

`src/Tool.ts` → `src/state` → `src/context` → `src/entrypoints/sdk`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

SOURCE READING · 1/2

OBSERVED

ToolUseContext는 port와 mutable projection이 함께 있다

ToolUseContext는 turn 실행에 필요한 capability bundle이지만 UI callback도 포함한다.

ToolUseContext 앞부분 src/Tool.ts:158-205

```
158  export type ToolUseContext = {
159    options: {
160      commands: Command[]
161      debug: boolean
162      mainLoopModel: string
163      tools: Tools
164      verbose: boolean
165      thinkingConfig: ThinkingConfig
166      mcpClients: MCPServerConnection[]
167      mcpResources: Record<string, ServerResource[]>
168      isNonInteractiveSession: boolean
169      agentDefinitions: AgentDefinitionsResult
170      maxBudgetUsd?: number
171      /** Custom system prompt that replaces the default system prompt */
172      customSystemPrompt?: string
173      /** Additional system prompt appended after the main system prompt */
174      appendSystemPrompt?: string
175      /** Override querySource for analytics tracking */
176      querySource?: QuerySource
177      /** Optional callback to get the latest tools (e.g., after MCP servers connect mid-query) */
178      refreshTools?: () => Tools
179    }
180    abortController: AbortController
181    readFileState: FileStateCache
182    getAppState(): AppState
… 18 lines omitted; use the source link for the complete range …
201      signal: AbortSignal,
202    ) => Promise<ElicitResult>
203    setToolJSX?: SetToolJSXFn
204    addNotification?: (notif: Notification) => void
205    /** Append a UI-only system message to the REPL message list. Stripped at the
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	options에 commands/tools/MCP가 있다.
그 표현이 만든 조건	AbortController와 read state가 있다.
그 표현이 만든 조건	get/set AppState port가 있다.
그 표현이 만든 조건	REPL 전용 JSX와 notification callback이 선택적으로 붙는다.

언어와 런타임은 여기까지 보장한다

구조적 typing은 여러 surface가 필요한 subset을 제공하게 하지만 optional field가 많아질수록 runtime capability check가 분산된다.

프로젝트가 이 메커니즘에 맡긴 책임

context가 composition 편의를 제공하는 동시에 core와 surface 경계를 흐린다.

경계를 잘못 읽을 때 생기는 위험

새 surface가 추가될 때 optional callback이 계속 늘어날 수 있다.

이 설명을 확인할 코드 src/Tool.ts:158-205

ALGORITHM A06 · 독자의 질문

state lifetime classification 은 어떤 시스템 문제를 닫는가

한 class 안의 field는 함께 보이지만 reset 시점·writer·지속 기간은 서로 다르다. 파일 단위 리팩터링만으로 묶으면 서로 다른 lifetime이 다시 결합된다.

코드가 필요해지는 이유

reset·persist·concurrency 규칙이 다른 state가 한 object에 섞이는 것을 방지한다.

OBSERVED — QueryEngine field 정의와 실제 read/write site가 각 state의 실질적 owner와 lifetime을 드러낸다.

INFERENCE — 이 알고리즘은 runtime 함수라기보다 refactoring 전에 state seam을 찾는 분석 절차다.

이 절에서 답할 결정 질문

field를 session·turn·request·derived state로 분류하면 어떤 구조 변경이 안전해지는가?

> ### 이 코드를 읽기 전에 잡을 개념
>
> **ownership과 lifetime.** 같은 object에 있다는 사실은 같은 lifetime을 뜻하지 않는다. 단일 writer, reset event, persistence destination을 함께 봐야 이동 가능한 경계가 생긴다.
>
> 184–207행의 field를 선언 위치가 아니라 write/read/persist site와 함께 추적한다.

이 설명을 확인할 코드 src/QueryEngine.ts:184-207

ALGORITHM A06 · 실제 코드

RECOMMENDATION

state lifetime classification 의 결정점은 어디에 있는가

리팩터링 전 각 state를 process·session·turn·tool-call·UI-frame 수명으로 분류한다.

입력 표현. field 정의, write site, read site, 그리고 persistence site

보존되는 작업 상태. field별 owner/lifetime table

session fields의 실제 예

src/QueryEngine.ts:184-207

```
184   export class QueryEngine {
185     private config: QueryEngineConfig
186     private mutableMessages: Message[]
187     private abortController: AbortController
188     private permissionDenials: SDKPermissionDenial[]
189     private totalUsage: NonNullableUsage
190     private hasHandledOrphanedPermission = false
191     private readFileState: FileStateCache
192     // Turn-scoped skill discovery tracking (feeds was_discovered on
193     // tengu_skill_tool_invocation). Must persist across the two
194     // processUserInputContext rebuilds inside submitMessage, but is cleared
195     // at the start of each submitMessage to avoid unbounded growth across
196     // many turns in SDK mode.
197     private discoveredSkillNames = new Set<string>()
… 7 lines omitted; use the source link for the complete range …
205       this.readFileState = config.readFileCache
206       this.totalUsage = EMPTY_USAGE
207     }
```

> 이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A06 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

field 정의, write site, read site, 그리고 persistence site은 처음부터 신뢰된 실행 상태가 아니다. field별 owner/lifetime table가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** 생성 시점을 찾는다. 이 결과가 다음 판단 "마지막 소비 시점을 찾는다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** 마지막 소비 시점을 찾는다. 이 결과가 다음 판단 "turn 경계를 넘는지 확인한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** turn 경계를 넘는지 확인한다. 이 결과가 다음 판단 "disk/remote projection 여부를 확인한다."의 전제가 된다.
결정 확정	**표현과 선택지를 좁힌다** disk/remote projection 여부를 확인한다. 이 결과가 다음 판단 "단일 writer와 concurrent reader를 기록한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** 단일 writer와 concurrent reader를 기록한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

모든 mutable field에는 한 primary owner가 있다. 수명보다 긴 cache는 invalidation 규칙이 있다. UI projection을 canonical state로 역수입하지 않는다.

ALGORITHM A06 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **구성·변형·성능의 질문으로 읽기.** 모든 mutable field에는 한 primary owner가 있다.
- **동시성·시간·합의의 질문으로 읽기.** 수명보다 긴 cache는 invalidation 규칙이 있다.
- **구성·변형·성능의 질문으로 읽기.** UI projection을 canonical state로 역수입하지 않는다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- closure에 숨은 state
- global singleton
- callback을 통한 간접 write

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 파일 경계보다 state lifetime을 우선해 seam을 정한다.

RECOMMENDATION ownership 표를 characterization test와 architecture decision record의 입력으로 사용한다.

ALGORITHM A06 · 운영과 현재 상태

작은 변환도 effect와 미확정 경계를 남긴다

분류표 자체는 effect를 만들지 않지만 이후 reset·copy·concurrency 정책의 입력이 된다.

실패 뒤 남는 상태

global singleton과 숨은 closure state는 선언만 검색한 표에서 보이지 않는다.

OBSERVED	QueryEngine에는 서로 다른 lifetime의 session 관련 field가 함께 존재한다.
CONDITIONAL	모든 read/write site를 추적했을 때에만 owner 분류가 구조 개선에 사용될 수 있다.
UNKNOWN	간접 callback과 feature-gated writer의 전체 집합은 선택된 줄만으로 확정되지 않는다.
NEXT READ	field 이동 전 symbol reference와 persistence path를 다시 검색하고 characterization test를 연결해야 한다.

다시 적용할 질문

1. 새 cache field는 turn reset과 session resume 중 어느 사건을 따라야 하는가?
2. writer가 둘이라면 single-writer 경계를 복원할 adapter는 어디에 있어야 하는가?

SELECTIVE EXECUTION TRACE

OBSERVED

새 progress event의 영향 분석

하나의 event가 여러 표면을 지나며 어디에서 의미를 잃을 수 있는지 본다.

책임 연결

Core

query가 typed event를 yield한다.

이때 남은 상태와 판단이 “REPL”에서 사용할 전제가 된다.

진입 경계

REPL

onQueryEvent가 state update로 투영한다.

이때 남은 상태와 판단이 “QueryEngine”에서 사용할 전제가 된다.

책임 연결

QueryEngine

SDKMessage로 변환하고 partial 포함 정책을 적용한다.

이때 남은 상태와 판단이 “Schema”에서 사용할 전제가 된다.

책임 연결

Schema

public union과 control protocol을 갱신한다.

이때 남은 상태와 판단이 “Remote”에서 사용할 전제가 된다.

책임 연결

Remote

serialization/backpressure/disconnect 의미를 정의한다.

이 지점에서 앞선 상태 변화가 외부에서 관찰 가능한 결과로 정산된다.

이 흐름 전체에서 보존해야 하는 것

같은 event identity와 terminal ordering을 모든 consumer가 보존해야 한다.

어디에서 설명이 틀어지는가

한 consumer가 unknown variant를 조용히 버리면 표면 간 의미가 갈라진다.

이 설명을 확인할 코드 src/entrypoints/sdk/coreSchemas.ts:1450-1508

DISTINCTION TABLE

RECOMMENDATION

공통화 대상과 surface 고유 책임

모든 코드를 한 engine으로 이동하는 것이 목표가 아니다.

공통 core	REPL 고유	SDK/headless 고유
turn state algebra	render/commit	wire schema
tool effect protocol	keyboard/modal	host callback
permission decision semantics	human prompt UX	remote decision timeout
transcript invariant	scrollback projection	backpressure/disconnect

이 표를 읽는 관점

중앙화할 것은 의미이고, 분리할 것은 표현과 transport다.

CONCEPT IN CONTEXT

RECOMMENDATION

권장 seam: ConversationRuntime

현재 두 shell 사이에 canonical state와 event를 소유하는 application port가 필요하다.

코드에서 읽어낼 수 있는 단서

출발 단서	SessionStatePort는 transcript와 read state를 소유한다.
연결되는 단서	TurnCoordinator는 query 입력과 terminal settlement를 소유한다.
연결되는 단서	REPL/SDK/print는 event projector와 interaction port만 제공한다.

이 단서들을 하나의 설계로 묶으면

이 구조는 QueryEngine 이름을 확대하는 것보다 현재 QueryEngine의 headless 가정을 명시하고 공통 층을 새로 정의하는 편이 안전하다.

시스템 이해에 남는 결론

실제 변경 전에는 REPL과 print의 결과 동등성을 characterization test로 고정해야 한다.

이 설명을 확인할 코드 src/QueryEngine.ts:175-183 src/screens/REPL.tsx:2793-2803

THEORY ATLAS · 1/2

용어를 외우지 않고, 어떤 보장을 설명하는 이론인지 묶어 읽는다

이 지도는 이 권의 알고리즘이 실제로 참조한 이론만 집계한다. 빈도는 중요도 점수가 아니라 여러 구현 결정을 같은 질문으로 비교할 수 있는 탐색 신호다.

상태·계산 모델

2개 알고리즘 · 2개 표현

입력이 어떤 중간 표현과 상태 전이를 거쳐 결과가 되는지, 그리고 각 전이 전후에 무엇이 계속 참이어야 하는지를 설명한다.

이 권에서 나타나는 방식. A03 우선순위 기반 CLI route resolution과 A05 print loop의 turn 간 상태 handoff에서 이 관점이 결정적인 이유는 "하나의 실행에서 정확히 한 top-level route만 선택된다. 또한 새 turn은 이전 message history를 본다."라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. chain of responsibility(A03), externalized state machine(A05)

스터디 질문. 단계의 순서나 표현을 바꿔도 같은 상태와 불변조건에 도달하는가?

동시성·시간·합의

2개 알고리즘 · 2개 표현

여러 실행이 같은 상태를 읽고 쓰는 동안 순서, 취소, 재시도, 관찰 시점이 결과의 의미를 바꾸지 않게 하는 조건을 다룬다.

이 권에서 나타나는 방식. A05 print loop의 turn 간 상태 handoff과 A06 state lifetime classification에서 이 관점이 결정적인 이유는 "새 turn은 이전 message history를 본다. 또한 모든 mutable field에는 한 primary owner가 있다."라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. versioned state(A05), single-writer principle(A06)

스터디 질문. stale read, 중복 실행, 취소가 끼어들어도 effect와 최종 상태의 의미가 보존되는가?

내구성·복구

1개 알고리즘 · 1개 표현

부분 기록, 재시작, 버전 변화가 있어도 이전 사실을 잃거나 같은 effect를 다시 확정하지 않도록 만드는 복구 계약을 설명한다.

이 권에서 나타나는 방식. A05 print loop의 turn 간 상태 handoff에서 이 관점이 결정적인 이유는 "새 turn은 이전 message history를 본다."라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. copy-merge(A05)

스터디 질문. 중단 지점에서 재개했을 때 무엇을 재생하고 무엇을 다시 실행하지 않아야 하는가?

괄호 안의 알고리즘 ID로 돌아가면 이론의 정의가 아니라 그 이론이 설명하는 source transition, 반례, 불변조건을 함께 확인할 수 있다.

THEORY ATLAS · 2/2

용어를 외우지 않고, 어떤 보장을 설명하는 이론인지 묶어 읽는다

이 지도는 이 권의 알고리즘이 실제로 참조한 이론만 집계한다. 빈도는 중요도 점수가 아니라 여러 구현 결정을 같은 질문으로 비교할 수 있는 탐색 신호다.

구성·변형·성능

3개 알고리즘 · 6개 표현

동일한 핵심 의미를 여러 실행 표면과 제품 변형에 조립하면서 비용과 의존성의 위치를 통제하는 방법을 다룬다.

이 권에서 나타나는 방식. A03 우선순위 기반 CLI route resolution과 A04 ask()의 one-shot engine adapter에서 이 관점이 결정적인 이유는 “하나의 실행에서 정확히 한 top-level route만 선택된다. 또한 한 ask 호출에는 한 engine instance가 있다.”라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. lazy loading(A03), partial evaluation(A03), adapter(A04), state handoff(A04), ownership(A06), lifetime(A06)

스터디 질문. 조립 방식이나 최적화를 바꿔도 핵심 계약이 adapter와 build 변형 안에서 보존되는가?

검증·관측·운영

1개 알고리즘 · 1개 표현

실행 결과를 관찰하고 회귀를 검출하며, cleanup과 운영 신호가 본래 동작을 왜곡하지 않도록 만드는 증거 체계를 다룬다.

이 권에서 나타나는 방식. A04 ask()의 one-shot engine adapter에서 이 관점이 결정적인 이유는 “한 ask 호출에는 한 engine instance가 있다.”라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. resource cleanup via finally(A04)

스터디 질문. 최종 출력뿐 아니라 실패·중단·복구 경로의 보장을 어떤 관찰과 테스트로 입증할 수 있는가?

괄호 안의 알고리즘 ID로 돌아가면 이론의 정의가 아니라 그 이론이 설명하는 source transition, 반례, 불변조건을 함께 확인할 수 있다.

ALGORITHM INDEX

OBSERVED

이 권의 결정적 알고리즘과 보장

알고리즘 이름보다 마지막 열을 먼저 읽는다. 프로젝트에서 중요한 것은 계산 기법 자체보다 그 계산으로 어떤 시스템 조건을 계속 참으로 만드는가이기 때문이다.

ID	문제와 구현	대표 불변조건
A03	**우선순위 기반 CLI route resolution** 하나의 binary에서 version, MCP host, remote, daemon, background, 일반 CLI를 충돌 없이 선택한다.	하나의 실행에서 정확히 한 top-level route만 선택된다.
A04	**ask()의 one-shot engine adapter** 기존 headless call site가 class lifetime을 직접 관리하지 않고 한 prompt를 실행하게 한다.	한 ask 호출에는 한 engine instance가 있다.
A05	**print loop의 turn 간 상태 handoff** 연속 stdin/command 입력을 독립 ask 호출로 처리하면서 conversation continuity를 유지한다.	새 turn은 이전 message history를 본다.
A06	**state lifetime classification** reset·persist·concurrency 규칙이 다른 state가 한 object에 섞이는 것을 방지한다.	모든 mutable field에는 한 primary owner가 있다.

SOURCE EVIDENCE INDEX · 1/2

OBSERVED

설명을 다시 코드로 되돌리는 출발점

보고서의 해석은 아래 고정 줄 범위에서 다시 검토할 수 있다. 링크는 모두 동일한 commit을 가리키며, 파일 이름만 비슷한 최신 branch로 이동하지 않는다.

고정 코드 범위	연결되는 설명	표현 방식
src/entrypoints/cli.tsx:28-93	우선순위 기반 CLI route resolution	algorithm
src/entrypoints/cli.tsx:95-180	Dynamic import가 architecture boundary 역할을 한다	code
src/main.tsx:240-310	main.tsx는 composition root이면서 결합 압력점이다	explain
src/main.tsx:3900-3940	main.tsx는 composition root이면서 결합 압력점이다	explain
src/replLauncher.tsx:1-22	launchRepl은 얇지만 REPL은 얇지 않다	code
src/screens/REPL.tsx:895-916	QueryGuard는 render state와 query lifecycle을 분리한다	code
src/utils/QueryGuard.ts:1-118	QueryGuard는 render state와 query lifecycle을 분리한다	code
src/screens/REPL.tsx:1182-1212	messagesRef와 React state는 같은 값의 서로 다른 시간 표현이다	code
src/screens/REPL.tsx:2584-2660	onQueryEvent는 event를 그대로 그리지 않고 보존 정책에 따라 축약한다	code
src/screens/REPL.tsx:4201-4245	useInput의 isActive와 propagation은 키보드 권한의 일부다	code
src/screens/REPL.tsx:2788-2804	영상의 “React 앱” 직관은 출발점이고 현재 코드의 결론은 더 복잡하다	explain
src/ink/renderer.ts:35-149	Custom Ink renderer는 Yoga tree를 reusable screen buffer로 투영한다	code
src/ink/output.ts:170-225	Custom Ink renderer는 Yoga tree를 reusable screen buffer로 투영한다	code
src/ink/ink.tsx:515-595	damage와 double buffer가 물리 화면 commit 범위를 결정한다	code
src/ink/ink.tsx:720-742	damage와 double buffer가 물리 화면 commit 범위를 결정한다	code
src/ink/renderer.ts:118-174	streaming token 하나가 화면에 보일 때	walkthrough
src/ink/ink.tsx:568-595	streaming token 하나가 화면에 보일 때	walkthrough
src/screens/REPL.tsx:2767-2804	REPL은 query()를 직접 호출한다	code
src/QueryEngine.ts:175-213	QueryEngine은 headless용 session abstraction이다	code
src/QueryEngine.ts:1249-1295	ask()의 one-shot engine adapter	algorithm
src/cli/print.ts:2140-2188	print loop의 turn 간 상태 handoff	algorithm
src/Tool.ts:158-205	ToolUseContext는 port와 mutable projection이 함께 있다	code
src/QueryEngine.ts:184-207	state lifetime classification	algorithm
src/entrypoints/sdk/coreSchemas.ts:1450-1508	새 progress event의 영향 분석	walkthrough

SOURCE EVIDENCE INDEX · 2/2

OBSERVED

설명을 다시 코드로 되돌리는 출발점

보고서의 해석은 아래 고정 줄 범위에서 다시 검토할 수 있다. 링크는 모두 동일한 commit을 가리키며, 파일 이름만 비슷한 최신 branch로 이동하지 않는다.

고정 코드 범위	연결되는 설명	표현 방식
src/QueryEngine.ts:175-183	권장 seam: ConversationRuntime	explain
src/screens/REPL.tsx:2793-2803	권장 seam: ConversationRuntime	explain

STUDY SYNTHESIS

상태 소유권 리뷰

읽은 내용을 요약하는 대신, 새로운 변경이나 장애 상황에서도 같은 보장을 찾아낼 수 있는지 확인한다.

검증 질문	REPL→query와 print→ask→QueryEngine→query를 각각 그린다.
검증 질문	messages, abort, usage, permission의 owner를 표에 채운다.
검증 질문	공통화할 의미와 남겨둘 surface 차이를 구분한다.

이 권의 완료 기준

“같은 엔진”이라는 문장 대신 정확한 공유 core와 다른 orchestration owner를 설명한다.

답에는 반드시 현재 snapshot의 코드 근거, 그 근거에서 도출한 해석, 아직 확인할 수 없는 부분을 구분해 적는다.